

AI 技術底層邏輯

全棧技術邏輯 · 與現有技術比對

這站的 AI 怎麼想 · 每層技術怎麼運作 · 跟業界/SOTA 比起來如何

專案：healing-studio (AI Director / director.today)。本文紮根 repo 實檔 (shared/ orb-*/agent-*/global-agent-*/server/services 生成與成本碼、drizzle schema) 描述真實底層邏輯，再對照 2026 業界/SOTA 替代方案。目的：把『黑箱 AI』講成可審視的工程；把技術選型講成『為什麼這樣選、跟別的比差在哪』。

一句話總綱：本站 AI 不是『丟給 LLM 直接答』，而是一條有守門的代理流水線--理解意圖 -> 產生計畫 -> 安全/成本雙閘 -> 批判精修 -> 排程執行 -> 驗證 -> 自省 -> 感知頁面真的變了沒 -> 必要時重新規劃。技術選型則一致偏向『型別安全、可回滾、免金鑰可降級、自建可控』。

1. 系統全貌 - 四殼一脊椎 + 代理流水線

分層（由上而下）：

層	技術	關鍵特性
前端殼層	React 19 + Vite + Wouter + tRPC client；四殼（video/social/learn/settings）一脊椎（Story Spine）	頁面狀態圖 PageAgent 可被 AI 驅動
API 邊界	tRPC v1.1（type-safe RPC）；37 router / 424 procedure；zod 輸入驗證	前後端共享型別，無手寫 API schema
AI 編排層	雙 orchestrator：client(DOM 派發) + server(LLM 工具計畫)；plan/safety/critic/cost/verify/reflect/perceive	本文 2 章主角
服務層	生成（fal/replicate/gemini）、LLM 路由門面、成本帳本、RAG/記憶、品質教練	server/services
資料層	Drizzle/MySQL（87 表）+ Pinecone 向量（halfvec 3072+HNSW，規劃遷 Supabase pgvector）	schema 無 FK 宣告，FK 由 migration 補
基建層	Railway（容器+cron）+ Cloudflare R2（媒體/備份，零 egress）+ Redis（生成鎖）	旗標化 strangler-fig 漸進上線

2. AI 底層邏輯 (核心)

2.1 LLM 路由與容錯 (不綁單一供應商)

auto 路由優先序：OpenRouter > Anthropic > Gemini > NVIDIA > Vertex > Forge。裸 claude-* 模型在未設金鑰/餘額不足時，由 inferEngineFromModelIdSafe 自動改寫成 OpenRouter 路徑 (anthropic/claude-*)，使用者零感知。normalizeModelForEngine 負責把 canonical id (帶前綴) 轉成各引擎認得的裸名 (AIDV-165 修：Perplexity 原生只認裸 sonar-pro)。

錯誤分級 classifyLLMError 是關鍵：把錯誤分成 transient (暫時，可重試) / permanent_model (模型無效，跳過重試立即回退) / quota / auth。permanent_model 不開斷路器、不誤把健康引擎熔斷 (AIDV-165)。配合 Cloudflare AI Gateway (AIDV-11) 做免費快取/觀測：重複問答命中快取 = 不再付 token。並行上限 MAX_CONCURRENT_LLM_CALLS (預設 5) 排隊、LLM_TIMEOUT 60s。

底層邏輯一句話：把『模型供應商』抽象成可替換、可降級、可觀測的門面，任何一家掛了不讓整個功能 500。

2.2 代理編排迴圈 - 9 階有守門的流水線

這是全站 AI 最核心的設計。刻意拆成兩個 orchestrator：client 半邊 (global-agent-orchestrator) 只驅動 DOM/頁面動作 (navigate/fillPrompt/setModel/applyPreset...)；server 半邊 (orbTaskOrchestrator) 跑 LLM 工具計畫 (API 呼叫/驗證/重試)。兩邊共用概念名 (step/retry) 但各司其職。完整一輪：

階段	運作 (repo 模組)	解決什麼
1 意圖	ai.chat 多步 planner 先抽 intent / 限制 / 引用 (orb-reasoning『思考步驟』面板可見)	不直接答，先理解
2 產計畫	LLM 產出 RunWorkflowAction (AgentPlan V3：steps + dependsOn + decisionMode + risk)	結構化、可審查
3 安全閘	agent-plan-safety：風險(low/med/high)/能力(hasCapabilityForPage)/已知動作 三檢；高風險要核准	擋越權/未知動作
4 批判精修	orb-plan-critic：LLM 二次審『draft->critique->refine』，抓未解 \${stepN.key}/工具參數漂移/缺前置/submit 無 prompt/依賴環	語意層把關
5 成本閘	orb-cost-governor：pre-flight 粗估 credits/時長/最高風險 tier，超預算先擋 (任何工具開火前)	先估後扣，紅線
6 排程	orb-dag-scheduler：拓撲分批 Promise.all，同 path 強制串接 (避免同頁 UI 競態)，偵測環則退回循序	防 UI race
7 執行+驗證	逐步派發 -> verifyToolResult 檢查工具輸出形狀/成功與否	工具結果可信
8 自省	orb-self-reflection：延遲過慢/輸出可疑過小/重複同款失敗 -> 結構化 verdict 寫回 orbMemory 供下輪學習	Reflexion 式
9 感知+重規劃	orb-perception-loop：執行後再抓一次 PageAgentSnapshot 與前一張 diff -> proceed/retry/replan/abort；重試耗盡才 reactive replan	關掉『以為成功其實沒變』缺口

這條流水線把學術上的 Plan-and-Execute + Reflexion + Grounded-Perception 三種範式合成在一個迴圈裡 (見第 5 章對映)。

2.3 記憶三層 (Agentic Memory)

層	實作	用途
Tier A 暫存	任務臨時草稿 (in-RAM + 選擇性 DB 持久化)	單次任務的中間狀態，做完即拋
Tier B 會話	會話/對話記憶 (in-RAM + RAG 索引 + per-user 摘要字串)	跨輪對話連貫；orbMemorySummary 凝練
Tier C 跨會話	持久語意記憶 (per-spirit 學習、語意 store)	跨 session 學習；精靈各自累積經驗

RAG 持久化失敗不可靜默：transient RAG outage 會讓那段時間寫入的記憶全部遺失，故 RAG 故障要顯式浮出 (非吞錯)。另有 ENABLE_RAG_INJECTION_GUARD 防『把外部檢索內容當指令執行』的 prompt injection。

2.4 RAG / 向量檢索

現用 Pinecone (PINECONE_INDEX=ai-director-memories)。已拍板向量後端 = halfvec(3072)+HNSW：3072 維對應大模型嵌入、halfvec (半精度) 省一半儲存、HNSW 給近似最近鄰高召回低延遲。規劃遷 Supabase pgvector (把向量併進主庫、退役 Pinecone 省月最低 \$50)。檢索 = 語意相似 topK 注入 prompt (RAG)，配合記憶三層形成『會話+長期』雙來源。

2.5 多精靈系統 (Multi-agent)

orb-agent-roles 定義約 92 個 role/persona (25 精靈為核心 + 專責子代理)，各有 system slice、能力、交接協議 (spirit-handoff-protocol)。global-agent-registry 是站級代理註冊表。設計重點：(1)每個精靈是『有界專責』而非萬能；(2)交接走顯式協議不亂搶；(3)per-spirit 記憶 (Tier C) 讓專長隨用累積。代價 = 多代理 = 成倍 token (見成本報告)。

2.6 生成管線 (估->原子扣->生成->驗證->落帳->關聯)

doPostGenComplete 是生成後處理樞紐：(1)寫 prompt_library (去重 upsert-by-content) (2)寫 digital_asset_library 資產(3)旗標 ON 時建 prompt<->asset junction (relation=derived) (4)落成本帳 (真實 costUsd)。成本原子性：先估 -> 原子扣點 -> 失敗全額退 (AIDV-160)。生成走 webhook+polling 雙路徑，靠 (promptId,assetId,relation) 唯一鍵與 idempotency 保證重跑不重複 (AIDV-158/163)。大檔上傳走 signed-URL 直傳 R2 (AIDV-15) 不過 Node。

2.7 品質與成本治理

qualityCoachEngine：對生成/腳本做品質評分與改進建議。orbCostGuard / cost-governor：執行前預算閘 + 執行後落帳對帳 (outbox 模式不漏帳、彙總淨額沖退款)。consent guard (AIDV-160)：選免費引擎絕不無聲回退付費並扣點 = 創作者紅線。

3. 全棧技術邏輯（每層：選什麼 · 怎麼運作 · 為什麼）

層	選用	運作	為什麼這樣選（repo 理由）
前端框架	React 19	fiber 並行渲染、Server Components 生態	成熟生態、團隊熟悉；19 的並行特性利於即時 UI
建置	Vite	ESBuild/Rollup、極快 HMR	比 webpack 快；dev 體驗好
路由	Wouter	~1.5KB hooks 式路由	刻意輕量；非 Next.js（不要 SSR/檔案路由的重量），54+ 路由自管
API	tRPC v11	端到端型別安全 RPC，前端直接呼叫後端型別	免手寫 OpenAPI/GraphQL schema；改後端型別前端即時報錯
ORM	Drizzle	SQL-first、輕、型別推導	比 Prisma 輕、無額外引擎；schema 不寫 FK，由 raw migration 補（可控）
DB	MySQL	成熟關聯庫	現況；規劃遷 Supabase(pg) 取 pgvector
向量	Pinecone -> pgvector	ANN 檢索	現用 Pinecone；遷 pgvector 併庫省成本
LLM 接入	OpenRouter + 直連	統一開道 + 可直連降級	一支金鑰打多家；可 fail-down 自建門面
生成	fal / replicate / gemini	逐模型 API	fal 主力（151 模型）；LoRA 走 fal、replicate 備援
認證	自建 JWT	自簽 token、script 雜湊、2FA TOTP	已拍板維持自建（不綁 Auth0/Clerk）；H4 硬化 fail-fast
鎖/併發	Redis SET NX PX	跨實例生成防重複	fail-open；未設 REDIS_URL 退記憶體版
狀態機	XState（收斂中）/自建	段落/鏡號狀態	規劃只留段落狀態機，其餘移除
部署	Railway	容器+cron+自動注入服務變數	非 serverless（有常駐 cron）；旗標化漸進上線
儲存	Cloudflare R2	S3 相容、零 egress	媒體+DB 備份；省 egress 是關鍵
監控	Sentry（選用）	錯誤/效能取樣	SENTRY_DSN 留空 = 不上報

4. 代理系統細節 (Agent System Internals)

4.1 動作型別 (AgentActionType) - AI 能對網站做的 16 種事

LLM 不是自由打字，而是只能產出『白名單動作』，每個動作型別有 zod schema 與風險等級。這是把 LLM 關進可控籠子的關鍵：

動作型別	作用	風險	備註
navigate / setTab / setMode	換頁/切分頁/切模式	low	純導航
setModality	切模態 (image/video/audio/voice/music/3d)	low	決定走哪條生成管道
fillPrompt / type	填提示詞/輸入	low	內容填充
setModel / setParam / applyPreset	選模型/調參數/套預設	low-med	影響成本 (貴模型)
toggleSetting / focusElement / openDialog / search / reset	切設定/聚焦/開窗/搜尋/重置	low	UI 操作
submit	送出生成 (真正花錢的動作)	med-high	過成本閘 + 確認門
runWorkflow	跑一整條多步工作流	high	DAG 排程 + 逐步驗證

風險推導 inferRiskLevelForAction + actionRequiresApproval：高風險 (submit/runWorkflow/刪除/權限) 一律要人類核准；low 動作可自動執行。這讓『AI 自動操作網站』在安全邊界內。

4.2 計畫結構 AgentPlan V3

一份計畫 = steps[] (每步：action + path + dependsOn + \${stepN.key} 引用前步結果) + decisionMode (決策模式) + 整體 riskLevel。step-ref 解析器把 \${step2.url} 這類佔位在執行期換成真值 (前步輸出餵後步輸入)。dependsOn 形成 DAG，交給 orb-dag-scheduler 拓撲分批；同 path 的步驟強制串接避免同頁競態。

4.3 兩個 Orchestrator 的分工

Orchestrator	負責	邊界
client : global-agent-orchestrator	驅動 DOM/頁面：navigate->settle 等待->awaiting_handler->dispatching->verify；換頁後等頁面 handler 就緒才派發	只碰 UI，不呼叫付費 API
server : orbTaskOrchestrator	跑 LLM 工具計畫：API 呼叫、verifyToolResult、retry/replan、寫 orbMemory	只碰後端，逐步驗證

為什麼拆兩半：UI 操作與 API 執行的失敗模式、重試語意、競態完全不同；混在一起會互相拖累。拆開後各自可測、可獨立 fail-safe。

4.4 精靈交接 (Spirit Handoff)

約 92 個 role/persona，交接走 spirit-handoff-protocol 顯式協議：當前精靈判斷任務超出自己職責，產生『交接意圖』（目標精靈 + 已知脈絡 + 未解問題）交給下一棒，而非亂搶或全部塞給一個萬能 agent。per-spirit 記憶 (Tier C) 讓每個精靈隨用累積專長。對映業界 = 結構化 multi-agent handoff (類 OpenAI Swarm / CrewAI 的 delegation，但自建可控)。

5. 生成系統與各管道細節 (Generation & Channels)

5.1 生成主接縫 (所有模態共用)

不論圖/影/音/語音，生成都走同一條接縫，確保成本與落帳一致：

階段	運作 (procedure / 服務)	保證
1 estimateCost	generate.estimateCost：先依模型+參數估點數 (modelPricing.estimatePoints)	先估
2 確認門	高成本/刪除/權限走 modal；常駐成本階梯儀表顯示	人類確認
3 原子扣點	先扣後生成 (atomic deduction)	防超花
4 submit	generate.submitStudioJob -> 供應商 (fal/replicate/gemini)	送單
5 取結果	webhook (FAL 簽章驗證+per-job token, AIDV-158) + polling 雙路徑	雙保險
6 recordGenResult	doPostGenComplete：寫 prompt_library(去重)+digital_asset_library+junction+成本落帳	落地
7 失敗處理	全額退點 (AIDV-160)；webhook+polling 靠唯一鍵冪等不重複	不重複扣

5.2 各模態管道 (Channels)

管道 (模態)	供應商 / 雙層策略	成本
圖像 image	fal flux/sd/ideogram、gemini imagen、vertex imagen (19 文生圖 + 17 圖生圖)	\$0.003–0.06/張
影片 video	雙層：草稿 Seedance/Kling/Wan -> 精修 Veo 3.1 (文生影 21 + 圖生影 21 + 影轉影 14)	\$0.10–0.80/支；最貴
語音 voice	ElevenLabs (家族需金鑰) + Qwen TTS (無金鑰)：22 TTS	~\$0.005–0.08
音樂/音效 audio	fal textToMusic (無 Suno 接點，已註明)：21 音樂音效	\$0.01–0.10
3D	文生 3D 5 + 圖生 3D 9	\$0.04–0.20
LoRA 訓練 training	fal 主、Replicate 備援 (需金鑰)：8 訓練	\$1–5/次
LLM 文字/推理	OpenRouter 門面 -> Anthropic/Gemini/... (11 LLM + 23 推理)	token 計

影片雙層是刻意設計：先用便宜草稿模型 (Seedance/Kling/Wan) 出多版讓使用者選，再對選中的用貴的 Veo 3.1 精修--省掉『一次就用最貴模型亂試』的浪費。這是成本與品質的工程折衷。

5.3 容錯/降級管道 (Fallback Chains)

管道	降級鏈	設計原則
LLM 管道	OpenRouter -> Anthropic -> Gemini -> NVIDIA -> Vertex -> Forge；裸 claude-* 自動改寫 OpenRouter	任一家掛不 500
研究管道	Perplexity Sonar (主) -> NewsData.io (備) -> 本地 fallback (第三級)	聯網研究不斷線
RAG 管道	向量檢索 (Pinecone) 失敗 -> 靜默回 [] -> LIKE 關鍵字 fallback 接手	RAG 是錦上添花非必需
生成取結果	webhook (即時) + polling (保底) 雙路徑	webhook 漏了 polling 補
生成鎖	Redis SET NX (跨實例) -> 未設/不可用 -> 記憶體版 / fail-open 放行	鎖故障不擋使用者

貫穿全站的鐵則：每條管道都『fail-down 不 fail-stop』--寧可降級成較弱結果，也不讓使用者撞 500。這是把『多供應商不穩定』這個現實，工程化成『使用者永遠有東西用』。

6. 與現有技術比對（選用 vs 業界替代）

每列：本站選用、業界主流替代、差異與取捨。判斷欄＝對本站情境的適配度。

層	本站選用	業界替代	差異與取捨	適配度
API 層	tRPC v1.1	REST / GraphQL / gRPC	tRPC＝端端型別、零 schema 重複、單體 TS 最省事；缺點：綁 TS、跨語言不友善。本站全 TS 故最佳適配	很適配
ORM	Drizzle	Prisma / TypeORM / Kysely	Drizzle 輕、SQL-first、無額外引擎程序、edge 友善；Prisma DX 較圓但較重、有 query engine。本站要可控 raw migration 故選 Drizzle	適配
路由	Wouter	React Router / TanStack Router / Next.js	Wouter ~1.5KB 極輕；Next.js 帶 SSR/檔案路由但重且改寫成本高。本站是 SPA 殼故選輕量	適配
向量庫	Pinecone(現)->pgvector	Qdrant / Weaviate / Milvus / pgvector	Pinecone 全託管省維運但月最低 \$50；pgvector 併進 pg 省錢但要自管索引。本站已拍板選 pgvector(halfvec+HNSW) 為成本	遷移中最佳
LLM 接入	OpenRouter + 門面	直連各家 SDK / LiteLLM / Vercel AI SDK	OpenRouter 一支金鑰多家、+5.5%；LiteLLM 自架省抽成但要維運。本站要省事+可降級故 OpenRouter + 自建門面	適配
代理框架	自建 (orb/global-agent)	LangGraph / CrewAI / AutoGen / OpenAI Agents SDK	自建＝完全可控、緊貼自家 UI 派發 + 成本開 + 頁面感知；框架＝起步快但抽象綁手、難塞自家守門。本站需求特殊故自建合理	自建可控
認證	自建 JWT	Auth0 / Clerk / Supabase Auth / NextAuth	自建＝無月費、無供應商鎖定、可控；託管＝省事但收費且綁定。已拍板維持自建 (H4 硬化 fail-fast)	已拍板自建
併發鎖	Redis SET NX PX	DB advisory lock / BullMQ / Postgres SKIP LOCKED	Redis 鎖跨實例快；BullMQ 適合任務佇列（規劃中耐久化要上）。現階段生成鎖選 Redis 對	適配
狀態機	XState(收斂)/自建	XState / Redux-Saga / 自建 reducer	XState 嚴謹但學習曲線；本站規劃只留段落/鏡號狀態機其餘移除＝避免過度工程	收斂中
部署	Railway	Vercel / Fly.io / Render / 自架 K8s	Railway 容器+cron+自動服務變數，適合有常駐 cron 的單體；Vercel 偏 serverless 不利常駐。本站有 5 支 cron 故 Railway 對	適配
物件儲存	Cloudflare R2	AWS S3 / Supabase Storage / Backblaze	R2 零 egress 是殺手鐮（媒體站 egress 最痛）；S3 egress 貴。本站媒體重故 R2 最省	最佳
DB	MySQL->Supabase(pg)	MySQL / Postgres / PlanetScale / Supabase	選 pg 為了 pgvector + Supabase 生態 (auth/storage/realtime 可選用)。注意遷移期雙跑成本	遷移規劃

7. 與 SOTA AI 代理範式對映

把本站自建代理迴圈，對映到 2026 學術/業界主流範式--它不是單一範式，而是多範式合成：

SOTA 範式	採用	本站如何體現	評語
ReAct (推理+行動交錯)	部分	planner 產 intent+plan 再行動；但本站把『推理』前置成完整計畫而非每步即興	Plan-first 比純 ReAct 更可審查
Plan-and-Execute	核心	先產整份 AgentPlan V3 再排程執行 = 典型 plan-and-execute	主骨架
Reflexion (自省學習)	有	orb-self-reflection 把失敗/慢/可疑輸出寫回 orbMemory 供下輪 planner 學習	跨輪改進
Critic / Self-refine	有	orb-plan-critic 在執行前 draft->critique->refine 兩段精修計畫	執行前攔錯
Grounded / World-model	有(特色)	orb-perception-loop 執行後抓頁面快照 diff 驗『真的變了沒』 = 以真實 UI 狀態為 ground truth	少見且關鍵
Tool-use / Function-calling	核心	16 種白名單動作 + zod schema = 受限 tool-use	安全 tool-use
Multi-agent / Handoff	有	~92 精靈 + 顯式 handoff 協議 + per-spirit 記憶	結構化委派
Cost-aware / Budgeted agent	有(特色)	cost-governor pre-flight 預算閘 + 原子扣 + 失敗全退	成本內建進迴圈，少見
MCP (模型脈絡協議)	規劃	BYOMCP 權限/稽核 (Wave 4) ；Railway 官方 MCP 待 OAuth	未來方向

總結：本站代理 = Plan-and-Execute 為骨 + Critic 與 Reflexion 為腦 + Grounded-Perception 為眼 + Cost-governor 為閘 + 受限 Tool-use 為手 + Multi-agent handoff 為團隊。其中『頁面感知驗證』與『成本內建守門』兩點，比多數開源代理框架更實戰。

8. 成熟度評估 · 優勢 · 風險

8.1 工程優勢

- [+] 型別安全一條龍 (tRPC+Drizzle+zod)：改後端型別前端即時報錯，重構成本低。
- [+] fail-down 哲學貫穿：LLM/研究/RAG/生成/鎖每條管道都有降級鏈，使用者極少撞 500。
- [+] 成本內建進代理迴圈 (pre-flight 閘 + 原子扣 + 失敗全退 + 落帳對帳)，少見且關鍵。
- [+] 頁面感知驗證 (perception loop) 關掉『AI 以為成功其實沒做到』的缺口。
- [+] 旗標化 strangler-fig 漸進上線：新功能躲旗標後預設 OFF，可秒回滾，prod 風險低。
- [+] migration 三鐵則 + journal 守門，資料庫變更不再卡死生產 (PO 教訓制度化)。

8.2 風險 / 技術債

- [!] 多代理 token 成本：~92 精靈 + 多階編排，每回合成倍 token；規模化前要壓 prompt cache/batch。
- [!] 供應商分散：8+ 生成/LLM 供應商，任一家 API/價格變動都要追（影片價以 fal 帳號頁為準）。
- [!] 遷移期雙跑：MySQL->Supabase 切換期兩庫並存 = 暫時雙倍 DB 成本與一致性風險。
- [!] 向量規模成本：halfvec(3072) 單筆~6KB，記憶條目成長則儲存/索引成本上升。
- [!] 自建代理維護成本：不靠框架 = 所有 planner/critic/verifier/scheduler 都要自己維護與測。
- [!] 耐久化未上：任務跨重啟存活 (BullMQ+Redis) 仍 Blocked (缺金鑰)，長任務目前有 5 分 timeout 風險。
- [!] 免費額度補貼 + 缺防濫用：50 免費生成 x 多帳號 = 成本破口，規模化前要設驗證/IP 門檻。

8.3 一句話定位

這是一套『工程紀律明顯高於一般

AI

side-project』的系統：選型偏保守可控、代理設計偏實戰（成本閘+頁面感知是亮點）、上線偏謹慎（旗標+可回滾）。主要代價是自建多代理的維護負擔與多供應商的協調成本；主要未竟是任務耐久化與成本優化。